

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 11

REST APIs & AJAX

11.1 REST APIs

In modernen Webapplikationen finden sich fast überall *Webservices*. Diese sind Schnittstellen, mit denen eine Maschine-zu-Maschine-Kommunikation stattfinden kann. An einem Ende – in der Regel auf einem Webserver – werden strukturiert Daten zur Verfügung gestellt, die dann von einer anderen Webapplikation (am selben oder auch auf einem anderen Server) konsumiert, d. h. weiterverarbeitet werden.

Konkret sichtbar wird dies, wenn eine Webapplikation eine strikte Trennung zwischen Frontend und Backend vornimmt, indem etwa unterschiedliche Technologien und Frameworks zum Einsatz kommen. So kümmert sich ein in PHP geschriebenes Backend etwa um die Verbindung zur Datenbank, der Speicherung, Abfrage und strukturierten Aufbereitung von Daten. Diese werden dann über einen Webservice, sprich eine definierte Schnittstelle zur Verfügung gestellt. Das Frontend – z. B. in Form eines JavaScript Frontend Frameworks wie Vue.js, Angular oder React – verbindet sich zu dieser Schnittstelle, und verwendet diese Daten, um damit das User-Interface der Webseite zu befüllen und die Informationen ansprechend und nutzer*innenfreundlich aufzubereiten.

Der Webservice arbeitet dabei plattformunabhängig, denn er gibt nur Daten in einem maschinenlesbaren Format wie JSON, XML oder auch einfachem Text aus. Diese Daten können von beliebigen anderen Applikationen konsumiert werden.

Bekannte Webservice-Technologien, -Protokolle bzw. auch -Architekturen sind XML-RPC¹, JSON-RPC², SOAP³ oder REST. Um letzteres dreht sich dieser Abschnitt.

11.1.1 Allgemeines zu REST

REST, kurz für *Representational State Transfer*, ist ein Paradigma oder auch Softwarearchitekturstil zur Erstellung von Webservices. Vergleicht man die Herangehensweise mit den zuvor aufgezählten Technologien, so fällt REST durch seine Einfachheit und das Verwenden bestehender Technologien auf. Während Technologien wie XML-RPC oder JSON-RPC es erlauben, Methoden einer über einen URI verfügbaren Webapplikation über das Netzwerk aufzurufen (RPC bedeutet „Remote Procedure Call“) und dadurch

¹<https://xmlrpc.com/>

²<https://www.jsonrpc.org/>

³<http://www.w3.org/TR/SOAP>

Daten zu erhalten geht REST hier einen anderen Weg: Es werden einfache HTTP- oder HTTPS-Requests mit den Methoden GET, POST, PUT, DELETE und HEAD an einen eindeutigen URI abgesetzt. Diese Aufrufe werden am Server verarbeitet und liefern die gewünschten Daten. Wie bei HTTP üblich, ist die Kommunikation damit beendet. Die Anfrage und die damit erhaltene Antwort muss so vollständig sein, dass sie für den*die Empfänger*in verständlich ist. Dies ist in den sechs REST-Prinzipien auch definiert.

Sechs Prinzipien einer REST-Architektur

REST ist keine standardisierte Technologie, sondern ein Architekturstil. Um zu definieren, wann ein Webservice, in weiterer Folge auch Schnittstelle oder API (Application Programming Interface) genannt, als REST API bezeichnet werden kann, wurden sechs Prinzipien festgelegt.

Client-Server: REST-Architekturen folgen dem Client-Server-Prinzip, das eine Trennung von Aufgaben vorsieht. Wie bereits in der Einleitung erwähnt, ist der Server für die Bereitstellung eines Dienstes und die Aufbereitung der Daten verantwortlich (z. B. Backend), welche von einem Client konsumiert und verarbeitet werden (z. B. Frontend).

Zustandslosigkeit: Da REST HTTP für die Anfragen und Antworten verwendet, zählt es auch zu den zustandslosen Protokollen. Daher muss die Antwort alle Informationen enthalten, um die Anfrage zu beantworten, da keinerlei Zustand (z. B. in Form von Sessions) gespeichert wird. Somit ist jede Anfrage an eine REST API unabhängig von vorangehenden oder nachfolgenden Anfragen.

Caching: Antworten eines REST-Aufrufs sollen zwischengespeichert werden können, sollen dies jedoch über die Header der Antwort kundtun.

Einheitliche Schnittstelle: Bei der Gestaltung der REST API gelten vier zentrale Anforderungen an die Schnittstelle:

- **Adressierbarkeit von Ressourcen:** Jede über die API zur Verfügung gestellte Ressource muss eindeutig identifizierbar sein. Dies geschieht über einen eindeutigen URI.
- **Repräsentationen zur Veränderung von Ressourcen:** Eine REST-API kann je nach Anforderung des Clients die Daten, die unter einer Ressource ausgeliefert werden, in verschiedenen Formaten (z. B. in JSON oder XML) zurückliefern.
- **Selbstbeschreibende Nachrichten:** Jede REST-Nachricht (die Antwort einer REST API) soll selbstbeschreibend sein, sodass klar ist, wie sie verarbeitet werden kann. Die nötigen Informationen dafür liefern die Metadaten der Antwort selbst: Der **Content-Type-Header** gibt das Inhaltsformat an (z. B. `application/json`), der Statuscode das Ergebnis der Anfrage (z. B. 200 oder 404). Der Client benötigt also keine zusätzlichen, externen Informationen, um eine Antwort korrekt interpretieren zu können.
- **Hypermedia as the Engine of Application State (HATEOAS):** Der REST-Client muss eine REST-Schnittstelle nur anhand der URLs, die vom Server zur

Verfügung gestellt werden, navigieren können. So kann in einer Antwort wiederum ein Link auf die nächste aufzurufende Ressource enthalten sein.

Mehrschichtiges System: Für den Client ist es nicht ersichtlich, ob er direkt zum Endserver verbunden ist, oder ob noch Systeme zwischengeschaltet sind. Die können etwa ein Load Balancer oder ein Proxy sein. Dies garantiert, dass das REST-System skalierbar bleibt.

Code on Demand (optional): Optional können REST-Schnittstellen auch ausführbaren Code wie z. B. JavaScript-Skripte mit übertragen, wenn dies notwendig ist.

11.1.2 Aufbau einer REST API

Eine REST-Schnittstelle ist also wie soeben definiert eine konkrete URI, die unter den verschiedenen HTTP-Methoden [2, Abschnitt 9] – allen voran GET, POST, PUT, DELETE und HEAD – Daten zur Verfügung stellt. Dabei haben die Methoden eine entsprechende semantische Bedeutung, die bei der Umsetzung beachtet werden sollte:

- **GET** ohne Angabe einer ID: Liefert eine Liste von Ressourcen (z. B. alle User*innen: GET `https://api.foo.com/users`).
- **GET** mit Angabe einer ID: Liefert eine spezifische Ressource mit einer ID (z. B. den*die User*in mit der bestimmten ID: GET `https://api.foo.com/users/3`).
- **POST**: Legt eine neue Ressource an (z. B. eine*n neue*n User*in: POST `https://api.foo.com/users`).
- **PUT** mit Angabe einer ID: Aktualisiert eine bestehende Ressource mit einer ID (z. B. aktualisiert den*die User*in mit dieser ID: PUT `https://api.foo.com/users/3`).
- **DELETE** mit Angabe einer ID: Löscht die Ressource mit einer ID (z. B. den*die User*in mit der ID: DELETE `https://api.foo.com/users/3`).
- **HEAD** mit oder ohne ID: Liefert Header-Informationen über eine Liste von Ressourcen oder eine einzige Ressource (z. B. die Größe der abzufragenden User*innenliste: HEAD `https://api.foo.com/users`).

Darüber hinaus werden auch die Methoden **OPTIONS** (Überprüfung, welche Methoden bei einer Ressource akzeptiert werden), **TRACE** und **CONNECT** unterstützt. Diese werden hier jedoch nicht weiter besprochen. Generell beziehen sich die in den weiteren Abschnitten angeführten Beispiele primär auf die Methoden **GET** und **POST**.

Datenformat bekanntgeben

Client und Server müssen einander mitteilen, welche Formate sie verarbeiten können (auf der Seite des Clients, denn dieser erhält die Antwort) und welche Formate tatsächlich geliefert werden (auf der Seite des Servers, denn dieser generiert die Antwort).

Dies geschieht für den Client bei der Anfrage über den HTTP-Header **Accept**, mit dem er mitteilt, dass er etwa Antworten in Plain-Text und JSON verarbeiten kann: **Accept: text/plain,application/json**.

Der Server schickt in seiner Antwort den Header **Content-Type** mit, in dem er den tatsächlich ausgelieferten Typ bekanntgibt: **Content-Type: application/json**.

Statuscodes einer REST-Antwort

In den Headerdaten der Antwort auf eine REST-Anfrage wird wie bei HTTP ein Statuscode mitgeschickt. Anhand dieses Codes wird dem Client mitgeteilt, ob die Anfrage erfolgreich war oder nicht. Hier können prinzipiell alle verfügbaren HTTP-Statuscodes [2, Abschnitt 15] verwendet werden, die gebräuchlichsten sind aber die folgenden:

- **200 OK:** Die Anfrage wurde erfolgreich ausgeführt. Ein Ergebnis wird mitgeliefert. Wird bei GET-Anfragen zurückgegeben.
- **201 CREATED:** Die Anfrage wurde erfolgreich ausgeführt und eine Ressource angelegt. Wird bei POST-Anfragen zurückgegeben.
- **204 NO CONTENT:** Die Anfrage wurde ausgeführt, es gibt jedoch keinen Inhalt, der zurückgegeben werden kann. Wird meist bei DELETE-Operationen verwendet, nachdem eine Ressource gelöscht wurde.
- **400 BAD REQUEST:** Genereller Fehlercode, wenn in der Anfrage ein Syntaxfehler enthalten ist und der Server diese nicht versteht (z. B. ID des*der User*in ist keine Zahl).
- **401 UNAUTHORIZED:** Authentifizierungsfehler. Die Ressource kann nur mit entsprechenden Credentials (d. h. einer Anmeldung) abgefragt werden.
- **403 FORBIDDEN:** Autorisierungsfehler. Der Client ist zwar angemeldet (im Gegensatz zur 401), besitzt aber nicht die Rechte, um diese Ressource abzufragen. Etwa wenn Admin-Rechte benötigt werden, man aber nur als User*in angemeldet ist.
- **404 NOT FOUND:** Die Ressource kann nicht gefunden werden. Entweder der URI existiert nicht oder die Ressource an sich (z. B. ein*e entsprechende*r User*in) ist nicht vorhanden.
- **406 NOT ACCEPTABLE:** Der Server kann keine Antwort erzeugen, die dem vom Client angeforderten Format entspricht.
- **500 INTERNAL SERVER ERROR:** Ein generischer, serverseitiger Fehler. Die Anfrage konnte nicht ausgeführt werden.
- **501 NOT IMPLEMENTED:** Die HTTP Methode für diesen URI ist nicht implementiert. Wenn etwa eine POST-Anfrage an eine URI geschickt wird, die aber nur GET implementiert.

Datenformate einer REST-Antwort

Die Antwort einer erfolgreichen Anfrage an eine REST API kann in verschiedenen *Datenformaten* erfolgen. Als Beispiel dient die Anfrage an die fiktive API `GET https://api.foo.com/users`, die alle User in einem System zurückgibt. Dies sind in diesem Beispiel jeweils drei Einträge.

XML: XML ist eines der am längsten verwendeten Markup-Formate, die zum REST-Datenaustausch zum Einsatz kommen. Es ist außerdem das namensgebende Format in AJAX (siehe 11.2) und ermöglicht es, komplexe Strukturen zurückzugeben und diese auch menschenlesbar aufzubereiten. Um mit PHP eine XML-Antwort zu erzeugen, können die in Vorlesung 10 demonstrierten Bibliotheken XMLWriter und DOM zum Einsatz kommen. Die Antwort kann wie folgt aussehen:

```
1 <users size="3">
2   <link rel="self" href="/users" />
3   <user id="1">
4     <username>jdoe</username>
5     <realname>John Doe</realname>
6     <link rel="self" href="/users/1" />
7   </user>
8   <user id="2">
9     <username>jane789</username>
10    <realname>Jane Doe</realname>
11    <link rel="self" href="/users/2" />
12  </user>
13  <user id="3">
14    <username>jimmy</username>
15    <realname>Jim Doe</realname>
16    <link rel="self" href="/users/3" />
17  </user>
18 </users>
```

Der Tag `<users>` leitet die Antwort ein und teilt in einem Attribut die Größe mit. Danach folgen `<user>`-Elemente, das Attribut `id` teilt die ID mit. Innerhalb des Elements werden die relevanten Informationen zurückgegeben. Die `<link>`-Elemente erfüllen das HATEOAS-Prinzip, damit aus der Antwort der URI für eine Detail-Abfrage gewonnen werden kann.

JSON: *JSON* ist ein einfaches, menschenlesbares Datenaustauschformat. Es wurde in der Vorlesung zu Web Fundamentals bereits im Detail besprochen. Der Vorteil von JSON gegenüber XML ist, dass JSON-Dokumente in der Regel kürzer und präziser sind, durch die Einteilung in Objekte und Arrays lassen sich Daten, die strukturiert vorliegen, sehr gut damit abbilden. JSON hat sich bei vielen APIs als Standard etabliert.

Eine Antwort in JSON kann wie folgt aussehen:

```
1 {
2   "size": 3,
3   "link": "/users",
4   "users": [
5     {
6       "id": 1,
7       "username": "jdoe",
8       "realname": "John Doe",
9       "link": "/users/1"
10    },
11    {
12      "id": 2,
13      "username": "jane789",
14      "realname": "Jane Doe",
15      "link": "/users/2"
16    },
17    {
18      "id": 3,
19      "username": "jimmy",
20      "realname": "Jim Doe",
21      "link": "/users/3"
22    }
23  ]
}
```

24 }

HTML: Die große Stärke von REST ist die Unabhängigkeit zwischen Ausgabeformat und darunterliegenden Daten. So kann auch HTML zurückgegeben werden, wenn dies für eine Applikation Sinn macht und der Client dies mit seinem **Accept**-Header auch als mögliches Format indiziert. Vor allem in Kombination mit AJAX (Abschnitt 11.2) ist dies nicht ungewöhnlich. Der Vorteil dabei ist, dass das HTML-Fragment schnell und ohne weitere Konvertierung etwa im Frontend eingebaut werden kann. Der Nachteil dabei ist, dass die verwendeten HTML-Elemente (und eventuell Klassen, IDs etc.) nicht für jedes Frontend geeignet sind. Dadurch leidet die Austauschbarkeit.

Eine HTML-Antwort kann etwa so aussehen. Da hier die Ausgabe direkt verarbeitet wird, wird auf die Links auf die jeweiligen einzelnen Userdaten (im Sinne von HATEOAS) verzichtet.

```

1 <div>
2   <div class="user" id="user1">
3     <p class="username">jdoe</p>
4     <p class="realname">John Doe</p>
5   </div>
6   <div class="user" id="user2">
7     <p class="username">jane789</p>
8     <p class="realname">Jane Doe</p>
9   </div>
10  <div class="user" id="user3">
11    <p class="username">jimmy</p>
12    <p class="realname">Jim Doe</p>
13  </div>
14 </div>

```

Text: Eine weitere Möglichkeit, Daten vom Server zu erhalten, ist einfacher *Text* ohne jegliches Markup. In diesem Fall wird der Text oft zeilenweise zurückgegeben, um ihn dann am Zeilenumbruch wieder trennen zu können. Ebenfalls möglich sind CSV-Formate, die nicht `\n` oder `\r\n`, sondern ein anderes Zeichen zum Trennen einzelner Daten verwenden.

Die User*innenliste kann – hier als einfaches CSV-Format – wie folgt aussehen:

```

1 1;jdoe;John Doe;/users/1
2 2;jane789;Jane Doe;/users/2
3 3;jimmy;Jim Doe;/users/3

```

11.1.3 Eine REST API in PHP implementieren

Das Grundprinzip einer REST API besteht also darin, Ressourcen unter einem eindeutigen URI in Kombination mit einer HTTP-Methode (GET, POST etc.) verfügbar zu machen und diese in einem bestimmten Format (oder mehreren) zurückzuliefern. Dies soll nun mit dem User*innenbeispiel, das bereits in Abschnitt 11.1.2 thematisiert wurde, praktisch umgesetzt werden. Um HTTP-Methode und URI einfach abfragen zu können, bietet sich ein Router an. In diesem Fall kommt **fhooe/router** unter Zuhilfenahme des **fhooe/router-skeleton** zum Einsatz. Da dieser Router sehr einfach gehalten ist,

werden jedoch nur die Methoden GET und POST demonstriert. Vollständige Router, wie in Abschnitt 6.2.4 gezeigt, unterstützen auch weitere Methoden. Dies wird auch in Vorlesung 12 anhand des Slim Frameworks demonstriert.

Als Ausgabeformat wird JSON (MIME-Type `application/json`) verwendet.

Definitionen der Routen

Zunächst werden in der Hauptdatei `index.php` (im Verzeichnis `public`) die Routen angelegt. Neben der Hauptroute GET `/`, welche der Startpunkt des Ablaufs ist, werden vier weitere Routen angelegt:

- GET `/users`: Listet alle User*innen im System auf.
- GET `/users/{id}`: Wird ein GET-Parameter `id` nach `/users` angegeben, wird nur der*die User*in mit dieser ID zurückgegeben.
- POST `/users`: Fügt eine*n neue*n User*in hinzu.
- GET `/addUser`: Zeigt ein Formular zum Hinzufügen eines*einer User*in an und ruft dann POST `/users` auf, um die eingegeben Daten an die API zum Eintragen zu übergeben.

Die Definitionen hierfür lauten wie folgt:

```
1 $router->get("/users", function () {
2     $userManager = new UserManager();
3     $userManager->listUsers();
4 });
5
6 $router->get("/users/{id}", function ($id) {
7     $userManager = new UserManager();
8     $userManager->listUser($id);
9 });
10
11 $router->post("/users", function () {
12     $userManager = new UserManager();
13     $userManager->addUser();
14 });
15
16 $router->get("/adduser", function () use ($latte) {
17     $latte->render("adduser.latte");
18 });
```

GET `/users`: Die Verarbeitung und Generierung der Ausgabe erfolgt in einem Objekt der Klasse `UserManager`. Durch Aufruf von `listUsers()` werden alle gespeicherten User*innen ausgegeben.

GET `/users/{id}`: Durch Angabe einer ID nach `/users` wird beim `UserManager`-Objekt `listUser($id)` aufgerufen, um den*die User*in mit dieser ID abzufragen und auszugeben. Das Error-Handling (z. B. bei nicht existenter oder nicht-numerischer ID) übernimmt die Klasse selbst.

POST `/users`: Auch hier wird ein Objekt der Klasse `UserManager` angelegt. In der Methode `addUser()` wird ein*e neue*r User*in hinzugefügt. Die Daten kommen dabei

aus der Payload des POST-Aufrufs und müssen unter dem Schlüssel `username` (in PHP als `$_POST["username"]` verfügbar) und `realname` (in PHP als `$_POST["realname"]` verfügbar) mitgeschickt werden. Dies geschieht hier über ein Formular unter der Route `GET /adduser`.

`GET /adduser`: Um POST-Daten erzeugen zu können, wird ein Formular angezeigt, welches User*innenname und echten Namen erfasst und danach die Router `POST /users` aufruft. Die Formularfelder haben die `name`-Attribute `username` und `realname`, wodurch in der POST-Payload die korrekten Einträge entstehen.

Alle User*innen abfragen: `listUsers()`

Die Methode `listUsers()` der Klasse `UserManager` gibt alle User*innen in einer JSON-Struktur zurück.

```
1 public function listUsers(): void
2 {
3     $query = "SELECT * FROM rest_api_example.user";
4
5     $statement = $this->pdo->query($query);
6     $users = $statement->fetchAll();
7
8     $jsonResponse = [
9         "size" => count($users),
10        "link" => $this->link
11    ];
12
13    foreach ($users as $user) {
14        $newUser = $user;
15        $newUser["link"] = "$this->link/{$user["id"]}";
16        $jsonResponse["users"][] = $newUser;
17    }
18
19    $this->showResponse(200, $jsonResponse);
20 }
```

Dazu werden zunächst mit einer Datenbankabfrage alle Zeilen und Spalten der Tabelle `user` abgefragt. Die Tabelle hat die Spalten `id` (der Primärschlüssel), `username` und `realname`. Das Ergebnis wird als assoziatives Array abgefragt (Option beim PDO-Objekt `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`).

Dann wird die JSON-Antwort erzeugt. Dazu werden in die Variable `$jsonResponse` die Einträge `size` und `link` hinzugefügt. Ersterer zählt die Anzahl der abgefragten Datensätze und setzt sie als Wert. Der Link (hier `/users`) wird in der Eigenschaft `$this->link` gespeichert und könnte auch direkt vom Router abgefragt werden (wenn dieser es unterstützt).

Schließlich wird über das Ergebnisarray der Datenbankabfrage iteriert. Jeder Eintrag mit den Positionen `id`, `username` und `realname` wird zunächst in der Variable `$newUser` gespeichert. Dann wird noch eine Position `link` für diesen einen User hinzugefügt. Ihr Wert besteht aus der Link-URL und der jeweiligen ID.

Ist das Array fertig konstruiert, wird `showResponse()` aufgerufen. Die Methode bekommt den Statuscode 200 für eine erfolgreiche Abfrage und das soeben erstellte Array `$jsonResponse` übergeben.

Ausgabe erzeugen: `showResponse()`

Diese Methode kommt immer zum Einsatz, wenn die REST API eine Antwort zurückgibt. Sie setzt den gewünschten HTTP- bzw. REST-Statuscode, den `Content-Type`-Header und gibt eine JSON-Struktur zurück, wenn es gewünscht ist.

```

1 private function showResponse(int $statusCode, ?array $content = null): void
2 {
3     $acceptHeader = $_SERVER["HTTP_ACCEPT"] ?? "";
4
5     if (str_contains($acceptHeader, $this->contentType) || str_contains($acceptHeader,
6         "*/")) {
7         http_response_code($statusCode);
8         header("Content-Type: " . $this->contentType);
9         if (isset($content)) {
10             echo json_encode($content, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES |
11                 JSON_UNESCAPED_UNICODE);
12         }
13     } else {
14         http_response_code(406);
15     }
16 }
```

Zunächst wird der HTTP-Header `Accept` abgefragt – abgesichert über den Null-Coalescing-Operator `??`, falls dieser Header gar nicht gesendet wird – und überprüft, ob der hier zurückgegebene Typ `application/json` (gesetzt im Konstruktor der Klasse in die Eigenschaft `$this->contentType`) darin enthalten ist. Ist dies nicht der Fall, weil der Client etwa gezielt XML über `Accept: application/xml` anfordert, dann wird der Statuscode 406 zurückgegeben. Dieser teilt mit, dass das gewünschte Format nicht erzeugt werden kann. Diese Prüfung ist allerdings sehr streng, weshalb zusätzlich geprüft wird, ob der `Accept`-Header den Wildcard-Wert `*/` enthält. Dieser bedeutet, dass der Client jedes beliebige Format akzeptiert – in diesem Fall wird die Antwort ebenfalls erzeugt. Das deckt gängige Clients wie `curl` oder `fetch()`-Aufrufe ohne explizit gesetzten `Accept`-Header ab (die `*/` senden) sowie Webbrowser, deren `Accept`-Header `*/` am Ende ihrer Präferenzliste enthält. Sind also JSON oder die Wildcard in der Header-Information enthalten, so wird die Ausgabe erzeugt.

Zunächst wird der übergebene Statuscode gesetzt, dann der `Content-Type`-Header auf `application/json`, denn dies ist der Antwort-Typ. Wurde nun ein Array im Parameter `$content` übergeben, wird dieses mit der Funktion `json_encode()` in eine JSON-Datenstruktur umgewandelt und mit `echo` ausgegeben.

Im Falle von `GET /users` entsteht dies bei drei hinterlegten User*innen. Dies deckt sich mit dem Beispiel aus Abschnitt 11.1.2.

```

1 {
2     "size": 3,
3     "link": "/users",
4     "users": [
5         {
6             "id": 1,
7             "username": "jdoe",
8             "realname": "John Doe",
9             "link": "/users/1"
10        },
11    ]
12 }
```

```

12     "id": 2,
13     "username": "jane789",
14     "realname": "Jane Doe",
15     "link": "/users/2"
16   },
17   {
18     "id": 3,
19     "username": "jimmy",
20     "realname": "Jim Doe",
21     "link": "/users/3"
22   }
23 ]
24 }

```

Anmerkung: Den `Accept`-Header mit `str_contains()` auf Inhalte zu vergleichen, ist eine schnelle und oft zu einfache Lösung. Mit `willdurand/negotiation`⁴ existiert etwa eine Komponente, die eine vollständige und professionelle Lösung implementiert.

Eine*n User*in abfragen: `listUser()`

Die Methode `listUser($id)` der Klasse `UserManager` gibt eine*n User*in mit gegebener ID in einer JSON-Struktur zurück.

```

1 public function listUser(int|string $id): void
2 {
3     if (is_string($id) && !is_numeric($id)) {
4         $this->showResponse(400, ["error" => "The given ID has to be a number"]);
5         return;
6     }
7
8     $query = "SELECT * FROM rest_api_example.user WHERE id = :id";
9
10    $statement = $this->pdo->prepare($query);
11    $params = [":id" => $id];
12    $statement->execute($params);
13    $users = $statement->fetchAll();
14
15    if (count($users) === 1) {
16        $jsonResponse = $users[0];
17        $jsonResponse["link"] = "$this->link/$id";
18        $this->showResponse(200, $jsonResponse);
19    } else {
20        $this->showResponse(404, ["error" => "No entry with ID $id found"]);
21    }
22 }

```

Zunächst wird überprüft, ob der Parameter – der eine Ganzzahl oder ein String (so kommt der Parameter aus der URL vom Router) sein kann – eine Zahl ist. Handelt es sich etwa um Buchstaben, wird eine Fehlermeldung mit dem Fehlercode 400 (Bad Request) und einer Fehlermeldung zurückgegeben. Handelt es sich um eine Ganzzahl oder eine Zahl im String wird fortgefahren.

Das `SELECT`-Statement ist ähnlich zu dem in `listUsers()`, nur wird auf die übergebene ID eingeschränkt. Das Ergebnis sollte daher immer ein*e User*in sein. Alternativ keine*r, wenn die ID nicht existiert.

⁴<https://packagist.org/packages/willdurand/negotiation>

Nach der Abfrage wird das Abfrageergebnis direkt in die Variable `$jsonResponse` gespeichert. Damit existieren die Positionen `id`, `username` und `realname`. Es wird noch `link` hinzugefügt und danach `showResponse()` aufgerufen.

Dies generiert diese Struktur für den User mit der ID 1:

```
1 {  
2   "id": 1,  
3   "username": "jdoe",  
4   "realname": "John Doe",  
5   "link": "/users/1"  
6 }
```

Ist die ID in der Datenbank nicht vorhanden, besteht das Abfrageergebnis nicht aus genau einem Eintrag (sondern aus null). In diesem Fall wird der Statuscode 404 zurückgegeben, weil die angeforderte Ressource nicht existiert.

Eine*n User*in hinzufügen: `addUser()`

Die Methode `addUser()` der Klasse `UserManager` erstellt eine*n User*in in der Datenbank.

```
1 public function addUser(): void  
2 {  
3   $query = "INSERT into rest_api_example.user SET username = :username, realname = :  
4     realname";  
5  
6   $statement = $this->pdo->prepare($query);  
7   $params = [":username" => $_POST["username"], ":realname" => $_POST["realname"]];  
8   $success = $statement->execute($params);  
9  
10  if ($success) {  
11    $this->showResponse(201, [  
12      "message" => "User successfully created",  
13      "link" => "$this->link/{$this->pdo->lastInsertId()}",  
14    ]);  
15  } else {  
16    $this->showResponse(500, ["error" => "An error occurred while creating the user"]);  
17  }
```

Zunächst wird ein Prepared Statement mit Username und Realname vorbereitet, mit den Parametern aus `$_POST` befüllt und ausgeführt. Das Ergebnis (erfolgreich oder nicht) wird in der Variable `$success` gespeichert. War das Einfügen erfolgreich, wird der Statuscode 201 für „Created“ zurückgegeben. Ist beim Einfügen in die Datenbank etwas schief gegangen, wird 500 ausgegeben. Dies weist auf einen internen Serverfehler hin.

Beim erfolgreichen Erstellen wird nur einfacher JSON-Output generiert. Wenn es gewünscht ist, kann hier etwa auch der soeben erstellte Datensatz wieder zurückgegeben werden.

11.1.4 Weitere Funktionalitäten von REST APIs

Die hier gezeigten Funktionalitäten zeigen nur einige wenige Dinge, die mit REST APIs erreicht werden können. Zentral sind etwa Authentifikation und Autorisierung: diese stellen sicher, dass bestimmte Endpunkte der API nur mit gültigen Credentials erreicht werden können und bestimmte Abfragen nur Ergebnisse liefern, wenn diese Credentials für die jeweilige Abfrage berechtigt sind.

Weiters sind selbstverständlich noch andere HTTP-Methoden ein wichtiger Bestandteil. Dies und mehr wird umfassend in Web Architectures (WAR3VO) besprochen und angewandt.

11.2 AJAX

In Abschnitt 11.1 wurde gezeigt, wie auf der Serverseite Daten strukturiert mit einfachen HTTP(S)-Aufrufen zur Verfügung gestellt werden können. Auf der Clientseite kommt dabei oft AJAX zum Einsatz, wenn es darum geht, Daten von einem Server abzufragen und im Frontend einer Webseite ohne Neuladen der Seite einzubauen.

11.2.1 Allgemein

AJAX steht für *Asynchronous JavaScript And XML* und beschreibt das Konzept des dynamischen Änderns von Webseiten mit Hilfe einer asynchronen Datenübertragung.

Synchronität vs. Asynchronität

Synchronität im technischen Sinn bedeutet, dass alle Abläufe in einem Softwareprogramm oder einer Webseite zeitlich gestaffelt ablaufen. Lädt die Webseite neue Daten, so wird gewartet, bis diese Daten empfangen wurden und erst dann wird der Rest der Seite geladen. Die Seite (oder auch das Programm) blockiert also solange alle weiteren Vorgänge, bis der aktuelle Vorgang abgeschlossen ist.

Von *Asynchronität* wird hingegen gesprochen, wenn solche Vorgänge quasi parallel ablaufen können. Die Webseite beginnt mit dem Laden von Daten und fährt danach gleich mit dem Anzeigen von anderen Dingen fort, die nicht auf diese Daten angewiesen sind. Ist das Laden der Daten beendet (der Zeitpunkt steht hier aber nicht fest), wird das System benachrichtigt und kann mit der Verarbeitung der Daten beginnen.

Asynchronität trägt in der Regel entscheidend dazu bei, dass (Web-) Applikationen reaktionsfreudig (engl. „responsive“) bleiben.

Das Prinzip von AJAX

AJAX ist keine eigene oder neue Technologie sondern die Verbindung bekannter Web-Technologien wie etwa JavaScript oder XML. Mit Hilfe dieser Technologien wird der klassische Ablauf einer Webanwendung verändert und dynamischer gestaltet. Die zentrale Veränderung ist dabei, dass der Seiten-Reload entfällt.

Klassischer Ablauf einer Webanwendung: Beim klassischen Schema fordert der Client mittels eines Requests am Server eine Seite an. Der Server sucht diese Seite in einem

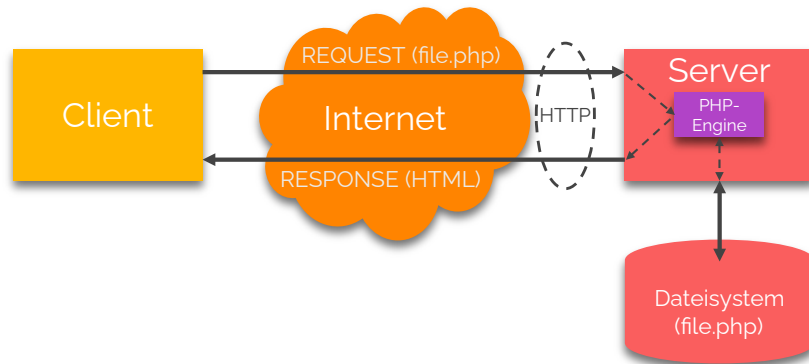


Abbildung 11.1: Bei einer klassischen Webanwendung (z. B. wie bisher rein in PHP) fordert der Browser eine Seite an, der Webserver verarbeitet diese und sendet den HTML-Code zurück.

Dateisystem, holt ggf. Informationen aus einer Datenquelle (z. B. Datenbank) und sendet den fertigen HTML-Code (ggf. mit Bildern, CSS etc.) an den Client zurück. Dies erzeugt den typischen Reload, da eine neue Seite angefordert wird. Abbildung 11.1 stellt dies schematisch dar.

Ablauf einer Webanwendung mit AJAX: Kommt nun AJAX zum Einsatz, so wird zwischen Client und Server noch eine Zwischenschicht eingeführt, die die Kommunikation mit dem Server übernimmt. Der Client schickt nun nicht mehr direkt einen HTTP-Request, sondern tätigt einen JavaScript-Aufruf. Dieser wird von der AJAX-Schicht verarbeitet, indem ein Request an den Server abgeschickt und die notwendigen Daten angefordert werden (z. B. wird eine einfache PHP-Datei oder eine REST API aufgerufen, welche die Daten abfragt und aufbereitet zurücksendet). Diese kommen vom Server nun nicht als zwingend als HTML retour, sondern oft auch in einem Datenformat wie XML (namensgebend), JSON, HTML oder als reiner Text, wie bereits in Abschnitt 11.1.2 gezeigt.

Die AJAX-Zwischenschicht nimmt diese Daten entgegen und kann nun die neuen Inhalte mittels JavaScript in die bestehende Seite integrieren. Auf diese Art und Weise wird das Neuladen der Seite vermieden. Zudem können auch nur einzelne Teile einer Seite ausgetauscht werden. Abbildung 11.2 veranschaulicht den AJAX-Ablauf.

Datenformate

Zur Übertragung der Daten vom Server zurück zur AJAX-Schicht kommen die Datenformate zum Einsatz, die bereits in Abschnitt 11.1.2 besprochen wurden:

- **XML:** Wenn die über AJAX am Server aufgerufene Ressource (z. B. eine REST API) XML zurück liefert, so muss natürlich zuvor ein XML-Format definiert worden sein. Einerseits muss die Datei am Server in diesem Format generiert werden, andererseits muss die AJAX-Schicht das Format kennen und parsen können, um die zurückgelieferten Daten auch verwenden zu können. Das Parsen der XML-Daten geschieht mit DOM-Methoden in JavaScript (`querySelector()`),

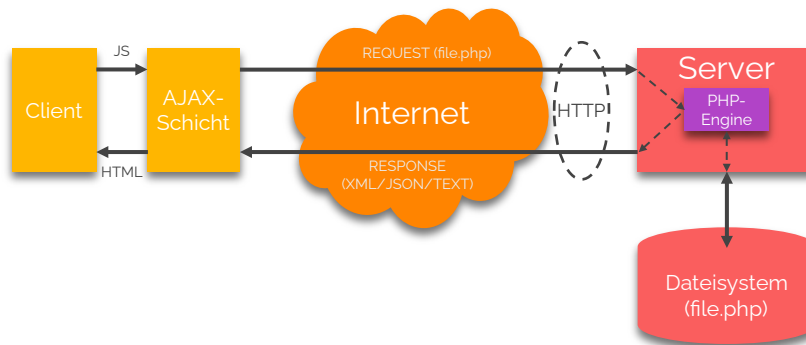


Abbildung 11.2: Bei einer AJAX-basierten Webanwendung kommuniziert der Browser über JavaScript mit einer Zwischenschicht. Diese setzt die eigentliche Anfrage zum Server ab (z. B. eine PHP-Datei oder REST API) und erhält Daten in XML, JSON oder einem anderen Format zurück. Diese Daten werden von der Zwischenschicht an den Browser retour gegeben bzw. eingebaut.

`getElementById()` etc.), denn diese funktionieren genauso mit XML-Dokumenten (wie auch schon in PHP in Vorlesung 10 demonstriert).

- **JSON:** Der Vorteil von JSON gegenüber XML ist, dass JSON-Dokumente in der Regel kürzer und präziser sind, durch die Einteilung in Objekte und Arrays lassen sich Daten, die strukturiert vorliegen, sehr gut damit abbilden. Ebenso ist JSON ein Subset von JavaScript, weshalb JSON-Objekte direkt als JavaScript-Objekte verstanden werden können, ohne zuvor geparsed werden zu müssen.
- **HTML:** Liefert der Server direkt *HTML-Code* zurück, so wird auch oft von *AHAH* (kurz für „Asynchronous HTML and HTTP“) gesprochen. Hierbei werden nie komplette HTML-Seiten zurückgeliefert sondern nur kleine Fragmente (z. B. `<article>`-Element oder ein Absatz). Diese werden dann mittels JavaScript (von der AJAX-Schicht) an der entsprechenden Stelle in der Webseite eingebaut. Diese Fragmente müssen am Server nicht zwingend dynamisch generiert werden (z. B. mittels PHP). Es kann auch einfach eine HTML-Datei angefordert werden, die genau dieses Fragment enthält.
- **Text:** Einfacher Text eignet sich dann sehr gut, wenn nur wenig Daten (etwa nur einzelne, kurze Strings) vom Server zurückgeschickt werden. Dadurch kann der Overhead des Parsens von Datenformaten entfallen.

11.2.2 Die Fetch API

Für die Kommunikation mit einem Server steht in modernem JavaScript die *Fetch API*⁵ zur Verfügung. Sie ist der native, in allen aktuellen Browsern verfügbare Standard, um asynchrone HTTP(S)-Requests abzusetzen und löst das ältere `XMLHttpRequest`-Objekt ab (siehe Abschnitt 11.2.6).

Das grundlegende Muster eines Fetch-Aufrufs besteht aus drei Bausteinen, die zunächst wie ein Rezept verwendet werden können. Das technische Fundament dahin-

⁵https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

ter – das Konzept der *Promises* – wird in der Lehrveranstaltung Web Architectures (WAR3VO) im dritten Semester ausführlich behandelt. Für die folgenden Beispiele genügt das Muster:

- **async**: Eine Funktion, die auf Daten aus dem Netzwerk wartet, wird mit dem Schlüsselwort **async** markiert.
- **await**: Ein Aufruf, dessen Ergebnis abgewartet werden muss, wird mit **await** versehen. Das Script „pausiert“ an dieser Stelle, bis das Ergebnis vorliegt, und liefert es dann direkt zurück. Dadurch liest sich der Code von oben nach unten, fast wie synchroner Code.
- **try/catch**: Tritt während des Requests ein Fehler auf, wird er im **catch**-Block abgefangen.

Ein vollständiger Fetch-Aufruf sieht damit grundsätzlich so aus:

```
1 async function loadData() {
2   try {
3     const response = await fetch("daten.php");
4
5     // Antwort enthält HTTP Status zwischen 200 und 299
6     if (response.ok) {
7       const data = await response.json();
8       // data weiterverarbeiten ...
9     } else {
10      // HTTP-Fehler (z. B. 404, 500) behandeln
11    }
12  } catch (error) {
13    // Netzwerkfehler behandeln (Server nicht erreichbar)
14  }
15 }
```

An diesem Muster fallen zwei Dinge auf, die für den korrekten Einsatz von Fetch zentral sind und im Folgenden kurz erläutert werden.

Zweimal await: `fetch()` wartet zunächst nur darauf, dass die *Antwort* des Servers eintrifft, also Statuszeile und Header. Der eigentliche *Inhalt* (der Body) wird erst durch einen zweiten Aufruf ausgelesen: `response.json()` für das Verarbeiten von JSON-Daten oder `response.text()` für reinen Text. Da auch das Auslesen des Bodys – gerade bei großen Datenmengen – Zeit in Anspruch nehmen kann, ist auch hier ein **await** notwendig. Man wartet also zuerst auf die Antwort, dann auf deren Inhalt.

HTTP-Fehler landen nicht im catch: Eine wichtige Besonderheit von Fetch: Ein HTTP-Fehlercode wie 404 Not Found oder 500 Internal Server Error gilt für `fetch()` *nicht* als Fehler. Die Anfrage war ja erfolgreich – der Server hat geantwortet, nur eben mit einem Fehlercode. Der **catch**-Block wird daher nur bei echten *Netzwerkfehlern* ausgelöst, etwa wenn der Server gar nicht erreichbar ist.

Um auf HTTP-Statuscodes zu reagieren, muss daher die Eigenschaft `response.ok` (**true** bei Statuscodes von 200–299) oder direkt `response.status` ausgewertet werden. Dies passt unmittelbar zur Trennung der Fehlerebenen in REST APIs aus Abschnitt 11.1.2: Der **catch**-Block deckt die *Transport*-Ebene ab (kam überhaupt eine Antwort

an?), die Prüfung von `response.ok` die *Applikations*-Ebene (war die Antwort inhaltlich erfolgreich?).

11.2.3 Beispiel: Textdaten verarbeiten

In diesem und in den Abschnitten 11.2.4 und 11.2.5 folgen nun drei Beispiele, welche die Verwendung von AJAX mit Hilfe der *Fetch API* illustrieren. Im ersten wird zunächst die grundsätzliche Verwendung mit einfachen Textdaten vom Server gezeigt. Neuer Inhalt wird auf einer HTML-Seite asynchron von einem Webserver angefordert und das Ergebnis eingebaut. Konkret befindet sich auf einer Seite eine Auswahlliste mit den neun Bundesländern Österreichs. Je nach Auswahl wird die entsprechende Landeshauptstadt auf der Seite angezeigt. Die Zuordnung von Bundesland zu Hauptstadt passiert dabei in einer PHP-Datei am Server, welche quasi als Datenquelle (wie eine REST API) dient. Diese erhält einen Parameter und liefert die entsprechende Stadt zurück, die dann angezeigt wird – alles ohne Neuladen der Seite.

Im Folgenden werden die einzelnen Bestandteile des Beispiels erläutert.

Die HTML-Seite

Ausgangspunkt ist eine einfache HTML-Datei. Diese enthält neben etwas Text ein Auswahlfeld (`<select>`) mit zehn Einträgen. Neben dem Standardwert „Please select“ sind die neun österreichischen Bundesländer erfasst. Die Liste hat die ID `state`, um sie später einfacher ansprechen zu können.

Darüber hinaus gibt es ein `<h2>`-Element mit der ID `capital`, in dem zunächst der Platzhaltertext „Select a state to have it replaced with its capital“ enthalten ist. Dieser Text wird durch die jeweilige Landeshauptstadt ersetzt werden.

Im `<head>` des Dokuments ist die Datei `ajax.js` mit `defer` eingebunden. In ihr findet der gesamte AJAX-Aufruf statt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>AJAX using Fetch and Text Data</title>
5     <meta charset="utf-8">
6     <script src="ajax.js" defer></script>
7   </head>
8   <body>
9     <h1>Show the Capital</h1>
10
11     <p>The following example retrieves the capital of the selected state
12       asynchronously and displays it in the heading.</p>
13
14     <h2 id="capital">Select a state to have it replaced with its capital</h2>
15
16     <form action="#">
17       <label for="state">Please select a state:</label>
18       <select id="state">
19         <option>Please select</option>
20         <option>Burgenland</option>
21         <option>Carinthia</option>
22         <option>Lower Austria</option>
```

```
23     <option>Styria</option>
24     <option>Tyrol</option>
25     <option>Upper Austria</option>
26     <option>Vorarlberg</option>
27     <option>Vienna</option>
28   </select>
29 </form>
30 </body>
31 </html>
```

Die JavaScript-Datei

Der Aufruf mit Fetch gestaltet sich einfach. Es genügt, die Auswahlliste abzufragen und ihr einen Event-Handler für den `change`-Event zuzuweisen. Jedes Mal, wenn ein neues Bundesland gewählt wird, ruft dieser die Funktion `sendRequest()` auf.

```
1 const select = document.getElementById("state");
2 select.addEventListener("change", sendRequest);
```

In der Funktion `sendRequest()` wird nun der Fetch-Aufruf an `states.php` abgesetzt. Da die Funktion auf Netzwerkdaten wartet, wird sie mit `async` markiert. Wie bei allen Funktionen, die von einem Event-Handler aufgerufen werden, wird das `event`-Objekt automatisch mitgegeben. Dieses besitzt das Unterobjekt `target`. Dies ist das Element, bei dem der Event ausgelöst wurde. Dieses Objekt besitzt wiederum die Eigenschaft `selectedIndex`, welche die Index-Position der Auswahl enthält. 0 ist dabei das erste, 1 das zweite Element, usw. Somit lässt sich der ausgewählte Index im Drop-down über `event.target.selectedIndex` ermitteln und als GET-Parameter `index` an den Aufruf von `states.php` angehängt.

Mit `await fetch(...)` wird der Request abgesetzt und auf die Antwort gewartet. Als Parameter wird ein Konfigurations-Objekt mit dem Schlüssel `headers` mitgeschickt. Dies informiert den Server, dass der Client mit Textdaten umgehen kann. Anschließend wird über `response.ok` geprüft, ob die Anfrage erfolgreich war. Falls ja, wird mit `await response.text()` der Antworttext ausgelesen und in das Element mit der ID `capital` gesetzt. Andernfalls wird eine Fehlermeldung angezeigt.

```
1 async function sendRequest(event) {
2   const index = event.target.selectedIndex;
3   const capital = document.getElementById("capital");
4
5   try {
6     const response = await fetch("states.php?index=" + index, {
7       headers: {"Accept": "text/plain"}
8     });
9
10    if (response.ok) {
11      capital.textContent = await response.text();
12    } else {
13      capital.textContent = "A problem occurred with requesting the selected capital.";
14    }
15  } catch (error) {
16    capital.textContent = "The server could not be reached.";
17  }
18 }
```

Im Vergleich zu einer Umsetzung mit `XMLHttpRequest` (siehe Abschnitt 11.2.6) entfällt bei Fetch das manuelle Vorbereiten, Konfigurieren und Abschicken des Requests über mehrere Methoden. Der gesamte Ablauf liest sich linear von oben nach unten.

Die PHP-Datei

Da AJAX ja verwendet wird, um vom Webserver dynamisch Daten im Hintergrund nachzuladen, muss es auch eine Instanz geben, die diese Daten entsprechend aufbereitet. Dies ist in vielen Fällen eine REST API, wie in Abschnitt 11.1 gezeigt, welche die benötigten Daten aus den Ressourcen (Datenbank, Dateien etc.) abfragt und entsprechend aufbereitet retour gibt. In diesem Fall ist es jedoch eine einfache, einzelne Datei `states.php`, mit einem Array als Datenquelle, das alle Landeshauptstädte sowie einen Standardeintrag zu Beginn enthält. Die Positionen decken sich hierbei mit denen aus der Auswahlliste.

Die PHP-Datei wird über die Fetch API aufgerufen und bekommt den GET-Parameter `index` mit übergeben. Zunächst wird mit `isset()` geprüft, ob dieser Parameter überhaupt vorhanden ist. Fehlt er, wird sofort Statuscode 400 (Bad Request) zurückgegeben, da ein wichtiger Parameter fehlt. Andernfalls wird mittels `array_key_exists()` geprüft, ob der übergebene Index im Array existiert, also zwischen 0 und 9 liegt. Wenn ja, wird Statuscode 200 (OK) gesetzt und die zugehörige Landeshauptstadt zurückgegeben. Existiert der Index nicht (etwa bei 10), wird Statuscode 404 zurückgegeben, da es unter diesem Index keinen Datensatz gibt.

Zuvor wird noch der Header `Content-Type` auf `text/plain` gestellt. Somit handelt es sich hierbei um reinen Text als Datenaustauschformat. Die PHP-Datei könnte diesen Wert auch in XML oder JSON verpackt retour geben. Da es sich aber nur um einen einzelnen Wert handelt, wäre der Overhead größer als der Nutzen daraus.

```
1 <?php
2
3 $capitals = [
4     "Please select a state",
5     "Eisenstadt",
6     "Klagenfurt",
7     "Sankt Pölten",
8     "Salzburg",
9     "Graz",
10    "Innsbruck",
11    "Linz",
12    "Bregenz",
13    "Vienna",
14 ];
15
16 header("Content-Type: text/plain");
17
18 if (!isset($_GET["index"])) {
19     http_response_code(400);
20 } elseif (array_key_exists($_GET["index"], $capitals)) {
21     http_response_code(200);
22     echo $capitals[$_GET["index"]];
23 } else {
24     http_response_code(404);
25 }
```

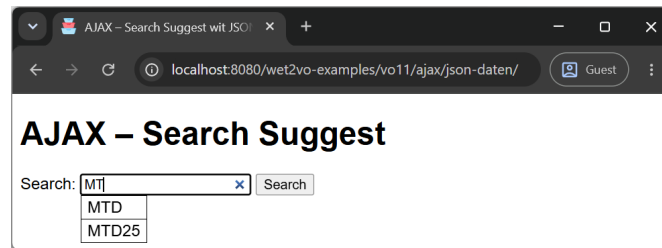


Abbildung 11.3: Nachdem die Buchstaben „MT“ eingetippt wurden, schlägt das System alle Wörter vor, die genau diese beiden Buchstaben an einer beliebigen Position enthalten.

11.2.4 Beispiel: JSON-Daten verarbeiten

Bei dem im Abschnitt 11.2.3 gezeigten Beispiel lieferte der AJAX-Aufruf ein Wort als reinen Text zurück. Im Folgenden soll nun ein Beispiel demonstriert werden, in dem die Rückgabewerte im JSON-Format vorliegen, wie dies bereits im Abschnitt 11.1 demonstriert wurde. Das Beispiel ist ein so genannter *Search Suggest*, d. h. bei der Eingabe von Buchstaben in ein Suchfeld werden alle Begriffe aus einem Datenbestand zurückgeliefert, in denen die eingegebenen Buchstaben vorkommen.

Prinzip

Ein Eingabefeld einer HTML-Datei wird als Suchfeld deklariert. Bei einem klassischen Ablauf wird beim Absenden des Formulars eine Suche in PHP durchgeführt und nach dem Reload der Seite werden die Suchergebnisse angezeigt.

Mit Hilfe von AJAX ist es nun möglich, bereits während der Eingabe des Suchbegriffs den Anwender*innen mögliche gültige Ergebnisse vorzuschlagen. Diese wissen dadurch bereits während der Eingabe, welche Begriffe ein Ergebnis liefern werden. Alternativ können genauso häufig verwendete Suchbegriffe vorgeschlagen werden (wie z. B. bei Google).

AJAX im Ablauf: Jedes Mal, nachdem ein Buchstabe getippt wurde, muss mittels AJAX nachgesehen werden, ob der bisher eingegebene Begriff (Needle) in einer Menge von definierten Wörtern (Haystack) gefunden wird. Wenn ja, wird der Begriff in einer Liste vorgeschlagen (siehe Abbildung 11.3).

Dieser gesamte Ablauf passiert im Hintergrund ohne Reload, also asynchron.

Die HTML-Datei

Die Datei `index.html` enthält das Formular. Das Eingabefeld erhält in der JavaScript-Datei `ajaxsearch.js` einen Event-Handler für den `keyup`-Event, der die zentrale Funktion des Search Suggests aufruft.

Das `<div>`-Element mit der ID `suggestions` ist zu Beginn leer und wird im Verlauf mit den möglichen Suchergebnissen befüllt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
```

```
4 <title>AJAX &ndash; Search Suggest wit JSON Data</title>
5 <meta charset="utf-8">
6 <link rel="stylesheet" href="styles.css">
7 <script src="ajaxsearch.js" defer></script>
8 </head>
9 <body>
10 <h1>AJAX &ndash; Search Suggest</h1>
11
12 <form id="form" method="get" action="output.php" autocomplete="off">
13   <div>
14     <label for="search">Search:</label>
15     <input type="search" id="search" name="search">
16     <button>Search</button>
17   </div>
18
19   <div id="suggestions"></div>
20 </form>
21 </body>
22 </html>
```

Die JavaScript-Datei

Auch hier wird zunächst das Suchfeld abgefragt und mit einem Event-Handler für den **keyup**-Event versehen. Jedes Mal, wenn eine Taste losgelassen wird, ruft dieser die Funktion `sendRequest()` auf.

```
1 const search = document.getElementById("search");
2 search.addEventListener("keyup", sendRequest);
```

In `sendRequest()` wird der aktuelle Suchbegriff über `encodeURIComponent()` für die Verwendung in einer URL aufbereitet (dies stellt sicher, dass für URLs problematische Zeichen durch ihre Escape-Sequenzen ersetzt werden) und als GET-Parameter **search** an `search.php` angehängt. Über das Konfigurationsobjekt wird angegeben, dass als Datentyp JSON erwartet wird. Da die Antwort hier (wie erwartet) im JSON-Format vorliegt, wird sie mit `await response.json()` direkt in ein JavaScript-Objekt umgewandelt. Das Ergebnis wird an `handleResponse()` übergeben.

```
1 async function sendRequest() {
2   const str = encodeURIComponent(search.value);
3
4   try {
5     const response = await fetch("search.php?search=" + str, {
6       headers: {"Accept": "application/json"}
7     });
8
9     if (response.ok) {
10      const data = await response.json();
11      handleResponse(data);
12    }
13  } catch (error) {
14    console.error("Request failed:", error);
15  }
16 }
```

Die Funktion `handleResponse()` erhält nun das fertig aufbereitete JSON-Objekt als Parameter. Falls der Eintrag `count` (die Anzahl der zurückgegebenen Einträge) größer als 0 ist, wird für jeden Eintrag ein `<div>` mit Event-Handlern für `mouseover`-, `mouseout`- und `click`-Events sowie einer CSS-Klasse erzeugt. Diese neu erzeugten Elemente werden dann in das bestehende `<div>`-Element mit der ID `suggestions` eingefügt.

```
1 function handleResponse(data) {
2   const suggestDiv = document.getElementById("suggestions");
3   suggestDiv.innerHTML = "";
4
5   if (data.count > 0) {
6     for (let i = 0; i < data.count; i++) {
7       const entry = document.createElement("div");
8       entry.addEventListener("mouseover", function () {
9         this.classList.add("suggestLinkOver");
10      });
11      entry.addEventListener("mouseout", function () {
12        this.classList.remove("suggestLinkOver");
13      });
14      entry.addEventListener("click", function () {
15        search.value = this.textContent;
16        suggestDiv.innerHTML = "";
17      });
18      entry.classList.add("suggestLink");
19      entry.textContent = data.words[i];
20      suggestDiv.appendChild(entry);
21    }
22  }
23 }
```

In den anonymen Funktionen der `mouseover`- und `mouseout`-Handler wird eine CSS-Klasse gewechselt, sobald sich die Maus über ein `<div>`-Element bzw. wieder hinaus bewegt. In der Funktion des `click`-Handlers wird der Wert des angeklickten Elements schließlich in das Eingabefeld gesetzt und das `<div>`-Element mit allen Vorschlägen wieder geleert.

Die PHP-Datei

In der PHP-Datei `search.php` wird zunächst eine Datenquelle (hier eine PHP-Klasse, die ein Array verwaltet) eingebunden. Von dieser Datenquelle wird nun ein Objekt erzeugt und darauf die Methode `search()` aufgerufen. Sie bekommt als Argument den als GET-Parameter übermittelten Sucheintrag. Ist dieser GET-Parameter nicht vorhanden, wird sofort der Statuscode 400 für „Bad Request“ zurückgegeben. Im Erfolgsfall ist es der normale Statuscode 200. Zuvor wird auch der `Content-Type`-Header auf `application/json` gesetzt.

```
1 $data = new SearchData();
2
3 header("Content-Type: application/json");
4 if (isset($_GET["search"])) {
5   http_response_code(200);
6   echo $data->search($_GET["search"]);
7 }
```

```

8 else {
9     http_response_code(400);
10 }

```

Die Methode `search()` läuft nun in einer Schleife durch das Array an hinterlegten Wörtern und überprüft mittels `mb_stripos()`, ob der Suchbegriff im Wort enthalten ist, wenn der Suchbegriff kein leerer String ist. Falls ja, wird es in das Array `$jsonResponse["words"]` geschrieben. Dieses assoziative Array wird später zur Eigenschaft `words` im JSON-Objekt. Nachdem alle Wörter durchlaufen wurden, wird das Array sortiert und mittels `count()` seine Länge ermittelt. Diese wird anschließend in `$jsonResponse["count"]` eingetragen. `$jsonResponse` wird dann mit Hilfe von `json_encode()` in ein JSON-Objekt verwandelt und mit `echo` zurückgegeben.

```

1 public function search(string $searchString): string
2 {
3     $jsonResponse["words"] = [];
4
5     if ($searchString !== "") {
6         foreach ($this->titles as $entry) {
7             if (mb_stripos($entry, $searchString, 0, "UTF-8") !== false) {
8                 $jsonResponse["words"][] = $entry;
9             }
10        }
11        sort($jsonResponse["words"]);
12    }
13
14    $jsonResponse["count"] = count($jsonResponse["words"]);
15
16    return json_encode($jsonResponse);
17 }

```

Als Datenbestand dient das folgende Array:

```

1 $this->titles = [
2     "Die FH",
3     "WET2V0",
4     "FH",
5     "Hagenberg",
6     "Datenbanken",
7     "Medientechnik und -design",
8     "MTD25",
9     "MTD",
10    "PHP",
11    "JavaScript",
12    "Österreich",
13    "Bla",
14    "Gut/Böse",
15 ];

```

Wird die Suche mit dem Begriff „MT“ ausgeführt, so wird folgender JSON-Output von `search.php` erzeugt:

```
{"words":["MTD","MTD25"],"count":2}
```

11.2.5 Beispiel: Formulardaten mit POST übertragen

Beide gezeigten Beispiele wurden mit der Methode GET übertragen. Gerade aber bei Formulardaten wird in der Regel die Methode POST benötigt, vor allem wenn nicht nur Textdaten, sondern etwa auch Dateien mit übertragen werden sollen. POST-Daten werden im Body des Requests mit übertragen und müssen daher über die Option `body` im Konfigurationsobjekt des `fetch()`-Aufrufs angegeben werden. Handelt es sich nur um Textdaten aus Eingabefeldern, kann als Wert grundsätzlich eine Zeichenkette in der folgenden Form übergeben werden:

```
1 fetch("details.php", {  
2   method: "POST",  
3   headers: { "Content-Type": "application/x-www-form-urlencoded" },  
4   body: "foo=bar&test=something",  
5 });
```

`foo` und `test` wären in diesem Fall die Namen zweier Formularfelder und `bar` und `something` deren Werte. Damit die Daten serverseitig im `$_POST`-Array ankommen, muss zusätzlich der `Content-Type`-Header auf `application/x-www-form-urlencoded` gesetzt werden – jenes Format, das auch ein HTML-Formular beim Absenden erzeugt.

Dieser händische Weg ist allerdings aus mehreren Gründen mühsam: Die Werte müssten zuvor aus den einzelnen Formularfeldern ausgelesen und zu dieser Zeichenkette zusammengesetzt werden, was bei vielen Feldern schnell unübersichtlich wird. Zudem müsste jeder Wert einzeln über `encodeURIComponent()` kodiert werden, da Sonderzeichen wie `&` oder `=` die Zeichenkette sonst zerstören würden. Und schließlich funktioniert dieser Ansatz ohnehin nur für reine Textfelder, nicht für Dateien.

Aus diesem Grund gibt es zur Aufbereitung von Formularinhalten für den Versand das `FormData`-Objekt⁶. Es nimmt einem das manuelle Zusammensetzen, das Kodieren und das Setzen des `Content-Type`-Headers vollständig ab und kann darüber hinaus auch Dateien übertragen.

POST-Requests mit Fetch und FormData

Zur Aufbereitung von Formularinhalten für den Versand dient das `FormData`-Objekt⁷. Sollen alle Felder eines Formulars berücksichtigt werden, wird das `<form>`-Element abgefragt und im Konstruktor übergeben. Wichtig ist, dass nur Formularfelder berücksichtigt werden, die ein `name`-Attribut aufweisen.

```
1 const form = document.querySelector("form");  
2 const formData = new FormData(form);
```

Bei Fetch wird das `FormData`-Objekt einfach als `body` im Optionen-Objekt des Requests übergeben. Die Methode wird über die Option `method` auf `"POST"` gesetzt. Anders als beim manuellen Versand einer Zeichenkette muss der `Content-Type`-Header hier *nicht* gesetzt werden: Übergibt man ein `FormData`-Objekt, erkennt der Browser dies und setzt den passenden `Content-Type` (`multipart/form-data`) automatisch – inklusive der technisch notwendigen `boundary`-Information, die die einzelnen Formularfelder im Body voneinander trennt. Dieser Header darf bei `FormData` sogar *nicht* manuell gesetzt werden, da die `boundary` nur dem Browser bekannt ist.

⁶<https://developer.mozilla.org/en-US/docs/Web/API/FormData>

⁷<https://developer.mozilla.org/en-US/docs/Web/API/FormData>


```
1 const response = await fetch("somefile.php", {  
2   method: "POST",  
3   body: formData,  
4 });
```

Auf der Serverseite stehen die Daten nun unter den entsprechenden Namen zur Verfügung: Textdaten in `$_POST` und Dateien in `$_FILES`.

Alternativ kann ein `FormData`-Objekt auch „manuell“ befüllt werden, falls nicht alle Inhalte des Formulars im AJAX-Request mitgeschickt werden sollen. In diesem Fall wird das Objekt ohne Argument angelegt, Einträge werden mittels `append()` hinzugefügt. Im folgenden Beispiel werden zwei Einträge angelegt:

```
1 const formData = new FormData();  
2 formData.append("foo", "bar");  
3 formData.append("test", "something");
```

Auch dieses `FormData`-Objekt wird dann der Methode `fetch()` übergeben.

Dateiuploads über FormData: Ein besonderer Vorteil des `FormData`-Objekts zeigt sich beim Hochladen von Dateien. Enthält das Formular ein Eingabefeld vom Typ `file` (siehe Abschnitt 7.3), so wird die ausgewählte Datei beim Erzeugen des `FormData`-Objekts automatisch mit übernommen und beim Fetch-Versand mitgeschickt. Serverseitig steht sie dann – wie beim klassischen Upload aus Abschnitt 7.3 – im gewohnten superglobalen Array `$_FILES` zur Verfügung. Die gesamte serverseitige Verarbeitung (Fehlercode-Prüfung, `move_uploaded_file()` usw.) bleibt also identisch. Der einzige Unterschied ist, dass der Upload nun asynchron und ohne Seiten-Reload erfolgt.

Ein wichtiger Kontrast zum klassischen Upload betrifft das `<form>`-Element selbst: Beim herkömmlichen Upload ist das Attribut `enctype="multipart/form-data"` zwingend erforderlich (siehe Abschnitt 7.3). Beim Versand über Fetch und `FormData` ist dies *nicht* nötig – der Browser setzt den passenden `Content-Type`-Header inklusive der nötigen `boundary`-Information beim Absenden automatisch und korrekt. Aus demselben Grund darf dieser Header bei `FormData` auch nicht manuell gesetzt werden.

Abschließend soll das Wissen der POST-Übertragung mit Hilfe eines `FormData`-Objekts verwendet werden, um ein einfaches Formular mit AJAX abzusenden und die Ergebnisse ohne Neuladen der Seite anzuzeigen.

Die HTML-Datei

Die HTML-Datei enthält hauptsächlich das `<form>`-Element, das zwei Formularfelder – einmal für Text und einmal für ein Passwort – enthält. Darüber hinaus existiert eine leere, ungeordnete Liste, in der die Ergebnisse eingebettet werden.

```
1 <!DOCTYPE html>  
2 <html lang="en">  
3   <head>  
4     <meta charset="UTF-8">  
5     <title>AJAX POST Request with FormData</title>  
6     <script src="postform.js" defer></script>  
7   </head>  
8   <body>  
9     <h1>AJAX POST Request with FormData</h1>  
10    <form action="#" method="post" id="registerForm">
```

```
11     <label for="user">Name:</label>
12     <input type="text" id="user" name="user">
13     <label for="pass">Password:</label>
14     <input type="password" id="pass" name="pass">
15     <button type="submit">Register</button>
16   </form>
17   <ul id="loginDetails"></ul>
18 </body>
19 </html>
```

Die JavaScript-Datei

In `postform.js` wird die gesamte Logik realisiert. Sie ist ähnlich aufgebaut wie in den vorangegangenen Beispielen. Das Formularfeld wird abgefragt und ein Event-Handler für den `submit`-Event angehängt. Tritt der Event auf, wird `sendRequest()` aufgerufen.

```
1 document.getElementById("registerForm").addEventListener("submit", sendRequest);
```

Die Funktion `sendRequest()` verhindert zunächst mit `event.preventDefault()` das klassische Absenden des Formulars (und somit einen Seiten-Reload). Danach wird aus dem Formular ein `FormData`-Objekt erzeugt und per `Fetch` an `details.php` geschickt. Bei erfolgreicher Antwort wird der zeilenweise zurückgegebene Text ausgelesen und verarbeitet. Beim Absenden wurde bereits mit dem `Accept`-Header kommuniziert, dass Text verarbeitet werden kann.

```
1 async function sendRequest(event) {
2   event.preventDefault();
3
4   const formData = new FormData(event.target);
5   const list = document.getElementById("loginDetails");
6   list.innerHTML = "";
7
8   try {
9     const response = await fetch("details.php", {
10       method: "POST",
11       headers: {"Accept": "text/plain"},
12       body: formData,
13     });
14
15     if (response.ok) {
16       const text = await response.text();
17       const entries = text.split("\n");
18
19       for (let i = 0; i < entries.length; i++) {
20         const item = document.createElement("li");
21         item.textContent = entries[i];
22         list.appendChild(item);
23       }
24     } else {
25       const item = document.createElement("li");
26       item.textContent = "A problem occurred with the request!";
27       list.appendChild(item);
28     }
29   } catch (error) {
30     const item = document.createElement("li");
```

```
31     item.textContent = "The server could not be reached.";
32     list.appendChild(item);
33 }
34 }
```

Zunächst wird die Liste hier zu Beginn jedes Aufrufs mit `list.innerHTML = ""` geleert. Damit wird verhindert, dass sich die Ergebnisse bei mehrmaligem Absenden immer wieder unterhalb anhäufen. Die PHP-Datei liefert kein JSON-Format, sondern einfach zeilenweise eine Zeichenkette im Format „Schlüssel: Wert“ für jeden Eintrag zurück. Um diese Daten zu verarbeiten, wird die in `text` gespeicherte Antwort an den Zeilenumbrüchen (`\n`) mit Hilfe von `split()` in ein Array aufgeteilt. Dieses wird durchlaufen und für jeden Eintrag ein `<li>`-Element erzeugt, welches in die ungeordnete Liste gehängt wird. Im Fehlerfall wird eine Fehlermeldung entsprechend ausgegeben.

Die PHP-Datei

Die PHP-Datei muss das soeben angesprochene Format erzeugen. Zunächst wird überprüft, ob in `_POST` Einträge vorhanden sind. Wenn nein, wird der Statuscode 400 zurückgegeben. Wenn ja, wird mit einer `foreach`-Schleife über alle Einträge in `$_POST` iteriert. Für jeden Eintrag stehen dabei der Schlüssel (`$key`) und der Wert (`$value`) direkt zur Verfügung, woraus jeweils ein String im Format `"key: $value"` gebildet und in das Array `$lines` geschrieben wird. Anschließend wird dieses Array mittels `implode()` in einen String umgewandelt, als Trennzeichen dient `PHP_EOL`, der Zeilenumbruch. Die Ausgabe wird mit `echo` und dem Statuscode 200 zurückgegeben. Die Antwort ist durch den `Content-Type-Header` als reine Textantwort gekennzeichnet.

```
1 header("Content-Type: text/plain");
2
3 if (!empty($_POST)) {
4     $lines = [];
5     foreach ($_POST as $key => $value) {
6         $lines[] = "$key: $value";
7     }
8     $output = implode(PHP_EOL, $lines);
9
10    http_response_code(200);
11    echo $output;
12 } else {
13    http_response_code(400);
14 }
```

Ausgabe und Anmerkungen

Bei Eingabe von „johndoe“ als Name und „topsecret“ als Passwort, werden diese Daten asynchron an die PHP-Datei geschickt. Diese liest diese aus dem `$_POST`-Array aus und gibt sie Zeilenweise zurück. Diese Rückgabe wird ohne Neuladen in die ungeordnete Liste eingefügt, sodass ein Ergebnis wie in Abbildung 11.4 entsteht.

Dieses Beispiel enthält keinerlei Validierung. Diese müsste in der PHP-Datei stattfinden, entsprechende Fehlermeldungen müssten mit der Response zurückgegeben und in JavaScript verarbeitet, d. h. angezeigt werden.

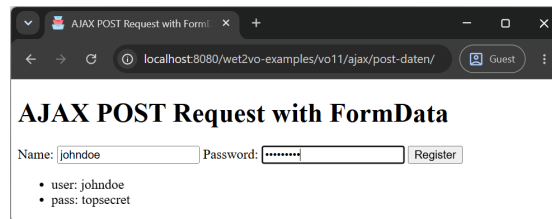


Abbildung 11.4: Nachdem Benutzer*innenname und Passwort eingegeben und abgeschickt wurden, werden diese, ohne die Seite neu zu laden, unterhalb in der Liste angezeigt.

11.2.6 Historischer Kontext: XMLHttpRequest

Bevor die Fetch API zum Standard wurde, war das *XMLHttpRequest*-Objekt (kurz *XHR*) über viele Jahre der einzige Weg, um in JavaScript asynchrone Requests abzusetzen. Es wurde ursprünglich von Microsoft entwickelt und später von den anderen Browserherstellern übernommen. Auf XHR geht auch der Name AJAX zurück: *A*ynchronous *J*avascript *A*nd *X*ML – auch wenn heute statt XML meist JSON übertragen wird.

Das Objekt ist nach wie vor in allen Browsern verfügbar und begegnet einem in älterem Code und Tutorials. Für neue Projekte wird es jedoch nicht mehr empfohlen, die Fetch API ist der moderne Ersatz.⁸ Zum Verständnis älterer Codebestände wird die grundsätzliche Arbeitsweise hier kurz am Bundesländer-Beispiel aus Abschnitt 11.2.3 gezeigt.

XHR ist *event-basiert*: Man legt ein Anfrage-Objekt an, bereitet den Request mit `open(method, url)` vor, registriert über `addEventListener()` einen Handler für den `load`-Event (der ausgelöst wird, sobald die Antwort vorliegt) und schickt den Request mit `send()` ab. Da die Antwort jederzeit eintreffen kann, wird die Verarbeitung in den Event-Handler ausgelagert – im Unterschied zum linearen Ablauf bei `async/await`.

```

1 const request = new XMLHttpRequest();
2
3 function sendRequest(event) {
4   request.open("GET", "states.php?index=" + event.target.selectedIndex);
5   request.addEventListener("load", handleResponse);
6   request.send();
7 }
8
9 function handleResponse() {
10  if (request.status === 200) {
11    document.getElementById("capital").textContent = request.responseText;
12  } else {
13    document.getElementById("capital").textContent = "A problem occurred.";
14  }
15 }
16
17 const select = document.getElementById("state");
18 select.addEventListener("change", sendRequest);

```

Im Vergleich zum Fetch-Beispiel aus Abschnitt 11.2.2 fällt auf, dass der Ablauf hier

⁸<https://developer.mozilla.org/docs/Glossary/XMLHttpRequest>

über zwei getrennte Funktionen verteilt ist (Absenden und Verarbeiten) und der Status manuell über `request.status` geprüft wird. Genau diese Verteilung über Event-Handler und das manuelle State-Tracking sind die Gründe, warum die modernere, sequenziell lesbare Fetch API XHR abgelöst hat.

Die WHATWG pflegt jedoch im Rahmen des *XMLHttpRequest* Living-Standards [9] weiterhin die Dokumentation und Funktionsweise des `XMLHttpRequest`-Objekts.

11.2.7 Ausblick: Promises

Das in diesem Kapitel verwendete `async/await`-Muster ist bewusst als Rezept eingeführt worden. Tatsächlich handelt es sich dabei um eine besonders gut lesbare Schreibweise für den Umgang mit *Promises*.

Ein *Promise* repräsentiert die finale Beendigung einer asynchronen Operation. Je nachdem, ob diese erfolgreich war oder fehlgeschlagen ist, kann ein entsprechender Callback aufgerufen werden. Historisch wurde dies über die Methoden `then()` (Erfolgsfall) und `catch()` (Fehlerfall) ausgedrückt:

```
1 fetch(url)
2   .then(response => response.json())
3   .then(data => {
4     // data verarbeiten
5   })
6   .catch(error => {
7     // Fehler verarbeiten
8   });
```

Dieser Code tut exakt dasselbe wie die `async/await`-Variante aus Abschnitt 11.2.2 – `await` ist lediglich eine neuere, sequenziell lesbare Schreibweise (eingeführt mit ES2017) für denselben Promise-Mechanismus. Beide Varianten liegen also *derselben* Fetch API zugrunde; sie unterscheiden sich nur in der Syntax. Das vollständige Konzept der Promises – Verkettung, Zustände und Methoden wie `Promise.all()` – wird in der Lehrveranstaltung Web Architectures (WAR3VO) im dritten Semester behandelt.

11.3 Serverseitige Konsumierung von REST APIs

Bisher wurde in diesem Kapitel gezeigt, wie REST APIs *bereitgestellt* werden und wie ein Browser als Client diese clientseitig über AJAX abfragt. In der Praxis müssen Webapplikationen jedoch häufig selbst auf externe Datenquellen zugreifen. Der eigene Webserver wird dabei zum Client, der Anfragen an eine fremde REST API stellt, die erhaltenen Daten (meist JSON) verarbeitet und in den eigenen Ablauf integriert.

Die zentrale Komponente für diesen Ablauf ist eine HTTP-Client-Bibliothek. Die PHP Framework Interoperability Group (PHP-FIG) definiert hierfür den Standard *PSR-18: HTTP Client* [6], der ein gemeinsames Interface für solche Clients festlegt. Die bekannteste Implementierung ist `guzzlehttp/guzzle`, die bereits in Vorlesung 5 für das Chuck Norris-Beispiel vorgestellt wurde.

Guzzle wird zunächst im Projekt mit Composer installiert:

```
1 composer require guzzlehttp/guzzle
```

11.3.1 Das Client-Objekt erstellen

Im ersten Schritt wird ein `Client`-Objekt angelegt, nachdem der Composer-Autoloader eingebunden wurde. Werden mehrere Requests an unterschiedliche Pfade derselben Domain gerichtet, kann im Konstruktor ein Basispfad (`base_uri`) gesetzt werden, sodass die einzelnen Requests nur noch den relativen Pfad angeben müssen.

```
1 use GuzzleHttp\Client;
2
3 require "vendor/autoload.php";
4
5 $client = new Client([
6     "base_uri" => "https://httpbin.org",
7 ]);
```

Das folgende Beispiel verwendet diesen Basispfad, um Requests abzusetzen. *httpbin.org*⁹ ist ein HTTP Request- und Response-Service, der eine REST API für Testzwecke bereitstellt und die gesendeten Daten zur Kontrolle wieder zurückgibt.

11.3.2 Requests senden

Requests werden über Shorthand-Methoden wie `get()` oder `post()` abgesetzt. Das zweite Argument ist ein assoziatives Array mit Optionen, etwa `query` für GET-Parameter oder `json` für einen JSON-Request-Body. Eine vollständige Auflistung bietet die Guzzle-Dokumentation¹⁰.

Im Folgenden werden ein GET- und ein POST-Request an den jeweiligen URL bei `httpbin.org` gesandt. Die Ergebnisse werden in zwei Response-Objekten gespeichert.

```
1 $getResponse = $client->get("/get", [
2     "query" => ["search" => "Hagenberg"],
3 ]);
4
5 $postResponse = $client->post("/post", [
6     "json" => ["username" => "jdoe", "realname" => "John Doe"],
7 ]);
```

11.3.3 Responses verarbeiten

Das Ergebnis eines Guzzle-Requests ist ein PSR-7 Response-Objekt. Über definierte Methoden können Statuscode, Header und Body ausgelesen werden:

- `getStatusCode()`: Liefert den Statuscode der Response.
- `getReasonPhrase()`: Liefert den textuellen Status.
- `getHeader($name)`: Gibt den Wert eines Headers zurück.
- `getBody()`: Liefert den Inhalt (Body) der Response.

Im folgenden Beispiel werden die Antworten des GET- und POST-Requests ausgewertet und ausgegeben. `httpbin.org` schickt als Antwort eine JSON-Datenstruktur mit den übermittelten Daten aus dem Request. Das Ergebnis ist in Abbildung 11.5 zu sehen.

⁹<https://httpbin.org/>

¹⁰<https://docs.guzzlephp.org/en/stable/request-options.html>

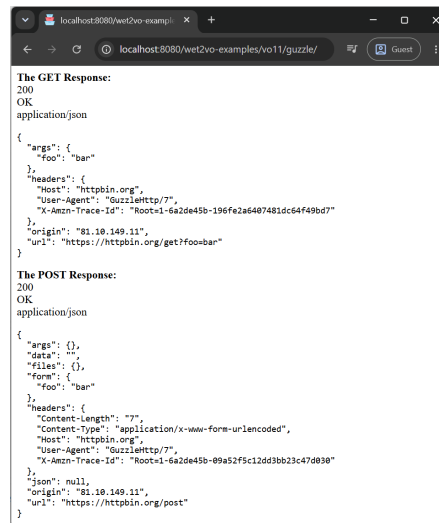


Abbildung 11.5: Die Antworten der Guzzle-Requests werden mit `echo` als Heredoc ausgegeben.

```

1 echo <<<GETRESPONSE
2 <p><strong>The GET Response:</strong><br>
3 {$getResponse->getStatusCode()}<br>
4 {$getResponse->getReasonPhrase()}<br>
5 {$getResponse->getHeader("Content-Type")[0]}</p>
6 <pre>{$getResponse->getBody()->getContents()}</pre>
7 GETRESPONSE;
8
9 echo <<<POSTRESPONSE
10 <p><strong>The POST Response:</strong><br>
11 {$postResponse->getStatusCode()}<br>
12 {$postResponse->getReasonPhrase()}<br>
13 {$postResponse->getHeader("Content-Type")[0]}</p>
14 <pre>{$postResponse->getBody()->getContents()}</pre>
15 POSTRESPONSE;

```

11.3.4 Fehlerbehandlung

Liefert eine Abfrage einen 400er-Fehlercode (Client Error, z. B. 404, weil die Ressource nicht existiert), so gibt Guzzle kein Response-Objekt zurück, sondern wirft eine `ClientException`. 500er-Fehlercodes (Server Error) lösen eine `ServerException` aus; die allgemeinere, beide Fälle abdeckende Exception ist `GuzzleException`.

Diese Exceptions werden mit einem `try/catch`-Block behandelt. Aus dem Exception-Objekt kann mit `getResponse()` die ursprüngliche Antwort des Servers abgefragt werden, um etwaige von der API mitgelieferte Fehlermeldungen auszuwerten:

```

1 try {
2     $getResponse = $client->get("/get", [
3         "query" => ["foo" => "bar"],
4     ]);
5 } catch (ClientException $e) {

```

```
6  $getResponse = $e->getResponse();
7  }
8
9  try {
10     $postResponse = $client->post("/post", [
11         "form_params" => ["foo" => "bar"],
12     ]);
13 } catch (ClientException $e) {
14     $postResponse = $e->getResponse();
15 }
```

Auf diese Weise existiert in jedem Fall ein verwertbares Response-Objekt, mit dem die Applikation weiterarbeiten kann.

500er-Fehlercodes werden mit einer `ServerException` bedacht, die allgemeinere (und beide Fälle abdeckende) Exception ist eine `GuzzleException`.

11.4 Weiterführende Literatur

Das Thema REST APIs wird in der PHP-Literatur etwas spärlich behandelt. [10] widmet ihm nur zwei der über 1.000 Seiten. Auch in [7] wird das Thema REST nur aus Client-Seite betrachtet.

Mit [5] existiert ein aktuelles Buch, das sich prinzipiell mit dem Design von APIs (abseits von PHP) beschäftigt.

Eines der ausführlichsten Werke zum Thema AJAX bietet sicherlich [3], wenngleich es das Buch sich in seinen über 900 Seiten etwas verliert und viele Dinge abdeckt, für Einsteiger*innen eher nebensächlich sind. Es ist außerdem schon etwas in die Jahre gekommen und arbeitet noch mit `XmlHttpRequest` und nicht der `Fetch API`.

Ein praxisbezogenes, in Deutsch gehaltenes Buch ist [4]. Hier wird das Thema AJAX zusammen mit einigen umgebenden Themen wie Sicherheit oder auch Usability ergänzt.

Auch in den großen JavaScript-Kompendien wie etwa [1, 8] finden sich Abschnitte zum Thema AJAX, die durchaus zur Einführung geeignet und viel aktueller als alle Spezialwerke sind.

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 11-31).
- [2] Roy T. Fielding, Mark Nottingham und Julian Reschke. *HTTP Semantics*. RFC 9110. IETF, Juni 2022, S. 1–194. DOI: 10.17487/RFC9110 (siehe S. 11-3, 11-4).
- [3] Anthony T. Holdener III. *Ajax. The Definitive Guide*. Sebastopol, USA: O'Reilly, 14. Feb. 2008 (siehe S. 11-31).
- [4] Kai Jäger. *Ajax in der Praxis. Grundlagen, Konzepte, Lösungen*. Heidelberg, Deutschland: Springer, 1. Dez. 2007 (siehe S. 11-31).
- [5] Arnaud Lauret. *The Design of Web APIs*. 2. Aufl. Shelter Island, USA: Manning Publications, 22. Juli 2025 (siehe S. 11-31).
- [6] PHP Framework Interoperability Group. *PSR-18: HTTP Client*. Hrsg. von Tobias Nyholm. Version 1.0.3. 23. Sep. 2023. URL: <https://www.php-fig.org/psr/psr-18/> (siehe S. 11-28).
- [7] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 11-31).
- [8] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 11-31).
- [9] Web Hypertext Application Technology Working Group. *XMLHttpRequest*. Living Standard. 11. März 2025. URL: <https://xhr.spec.whatwg.org/> (siehe S. 11-28).
- [10] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 11-31).